

Iceberg Ahead: A Machine Learning Adventure



Introduction to Machine Learning

Predict who survived the Titanic. Build your first ML model

What is Machine Learning?

Machine learning is a branch of artificial intelligence where computers learn patterns from data rather than following explicitly programmed rules.



Traditional Programming

Explicit rules defined by humans.

Input + Rules → Output



Machine Learning

Patterns learned from examples.

Input + Output → Rules

A Brief History

- 1950 Alan Turing proposes the "Turing Test" — can machines think?
- 1957 Frank Rosenblatt creates the Perceptron — first neural network
- 1967 Nearest Neighbour algorithm — early pattern recognition
- 1979 Stanford Cart — first autonomous vehicle navigates a room
- 1995 Tesco Clubcard — UK's first data-driven retail loyalty scheme
- 1997 IBM Deep Blue beats world chess champion Garry Kasparov
- 2006 Geoffrey Hinton coins "deep learning" — neural networks go deeper
- 2012 AlexNet wins ImageNet — deep learning revolution begins
- 2016 AlphaGo defeats world Go champion Lee Sedol
- 2025 Enablis develops pricing engine for Great Rail Journeys

Types of Machine Learning



Supervised Learning

Learn from labelled examples – we know the "right answer".

Example: Predicting survival from passenger data.



Unsupervised Learning

Find patterns in unlabelled data – no "right answer" provided.

Example: Customer segmentation by behavior.



Reinforcement Learning

Learn through trial and error – rewarded for good decisions.

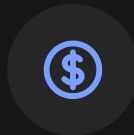
Example: Training a robot to walk.

Real-World Applications

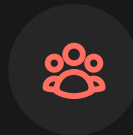
How Enblis leverages Machine Learning to drive business impact across industries.



Retail



Pricing



Propensity



Deep Dive: Retail & Price Modelling

Retail Analytics

Predict customer behaviour and optimise inventory.

- **Demand forecasting:** Units sold next week
- **Recommendations:** "Customers also bought..."
- **Churn prediction:** Who is likely to leave?

Impact: Reduced stockouts, +Sales

Price Modelling

Dynamically set prices based on demand and competition.

- **Airline pricing:** Changes based on time
- **Surge pricing:** Real-time demand (Uber)
- **Insurance:** Risk-based premiums

Impact: Maximised revenue & margins

Propensity Modelling & Other Apps

User Propensity Modelling

Predicting the likelihood of a user taking a specific action (Buy, Churn, Click, Default).

Impact: Targeted marketing & better risk management.

DOMAIN

APPLICATION

Healthcare

Disease diagnosis, drug discovery, readmission prediction

Finance

Fraud detection, credit scoring, algorithmic trading

Manufacturing

Predictive maintenance, quality control

Transport

Route optimisation, autonomous vehicles

The ML Workflow (Stages 1-3)

1

Data Exploration

Load data, check missing values, spot patterns.
"Garbage in = garbage out."

2

Feature Engineering

Create new features from raw data. Transform formats.
This is where the magic happens.

3

Train/Test Split

Divide data (80/20).
Evaluate on unseen data to avoid memorisation (overfitting).

⚠️ KEY INSIGHT: Understand Your Data First!

Understanding the true meaning of your data saves hours of rework. Ask: Has usage changed over time? What's the data quality? Are there hidden biases?

The ML Workflow (Stages 4-6)

4

Model Building

Algorithm: e.g. XGBoost. Try multiple models, tune hyperparameters, and select the best.

5

Model Evaluation

Run predictions on test set. Measure Accuracy, Precision, and Recall scorecard.

6

Iterate & Improve

Analyse errors. Try new features. Repeat until satisfied. ML is iterative.

DON'T LET YOUR MODEL CHEAT!

Always evaluate on data the model has never seen. Split your data before any processing. If you tune on test data, you're fooling yourself – not improving your model.

Feature Engineering Example

Creating new features from existing data to improve model performance (Retail: Customer Churn Prediction)

Raw Transaction Data

Purchase dates (customer #1042):

2024-01-15, 2024-02-20, 2024-03-10

2024-05-22, 2024-08-01 (last)

Order amounts:

£45.99, £120.50, £32.00, £89.99, £55.25

Product categories:

Electronics, Clothing, Home



Engineered Features

DaysSinceLastPurchase

Today - LastPurchaseDate = 180

PurchaseFrequency

Orders / Months active = 0.7/mo

AvgOrderValue

Sum(orders) / Count = £68.75

CategoryDiversity

Unique categories = 3

💡 RFM (Recency, Frequency, Monetary) features are classic retail predictors. Good feature engineering uses domain knowledge to reveal patterns.

Hyperparameters

Settings you choose before training – like adjusting the "knobs" on your model

Decision Tree

Like a flowchart of yes/no questions. "Is age > 30?" → "Is income > £50k?" → Prediction

Hyperparameter: max_depth

How many questions deep?

Shallow (3) = simple, general

Deep (20) = complex, specific

Random Forest

Many decision trees that "vote" together. Like asking 100 experts and going with the majority.

Hyperparameter: n_estimators

How many trees to grow?

Few (10) = fast but less stable

Many (500) = slower but robust

Neural Network

Layers of connected "neurons" that learn patterns, loosely inspired by the brain.

Hyperparameter: learning_rate

How big are the learning steps?

High (0.1) = fast but may overshoot

Low (0.001) = slow but precise

Why does this matter?

The same algorithm with different hyperparameters can give very different results. Finding the right settings is called "tuning" – usually done by trying different values and seeing what works best on your validation data.

Visualisation Techniques

Confusion Matrix

Grid showing predictions vs actual outcomes. Look for high numbers on the diagonal.

Actually Yes → Predicted
Yes ✓

ROC Curve & AUC

Summarises model quality. 0.8 - 0.9 is Good, 1.0 is Perfect model.

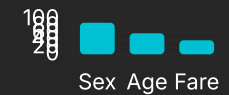


Prediction Probabilities

Histogram of confidence. Ideal: clear separation between classes.

Feature Importance

Ranked list of which features drive the predictions most.



Understanding the Confusion Matrix

A table showing how well your classifier performs – comparing predictions vs actual outcomes.

		PREDICTED		TOTAL
		YES	NO	
ACTUAL	YES	100 True Positive (TP) ✓	5 False Negative (FN) Type II	105 actual yes
	NO	10 False Positive (FP) Type I	50 True Negative (TN) ✓	60 actual no
TOTAL		110 predicted yes	55 predicted no	165 TOTAL

Row totals = actual counts | Column totals = predicted counts

Key Metrics (using row/column totals)

Accuracy = $(TP + TN) / \text{Total}$

$(100 + 50) / 165 = 91\%$

How often is the classifier correct?

Precision = $TP / \text{Predicted YES column}$

$100 / 110 = 91\%$

When it predicts YES, how often correct?

Recall = $TP / \text{Actual YES row}$

$100 / 105 = 95\%$

When actually YES, how often detected?

Specificity = $TN / \text{Actual NO row}$

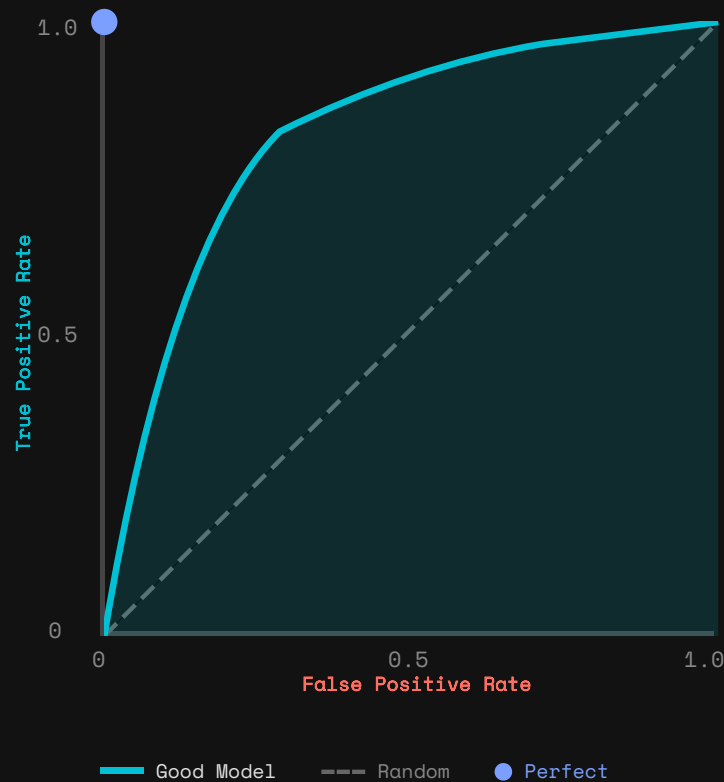
$50 / 60 = 83\%$

When actually NO, how often detected?

Tip: Row totals → Recall | Column totals → Precision

ROC Curve & AUC

Receiver Operating Characteristic – visualizing classifier performance across all thresholds.



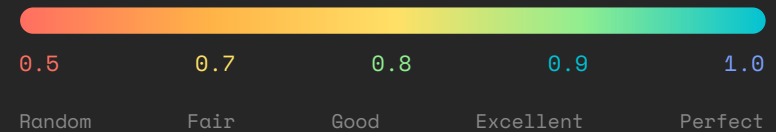
What is the ROC Curve?

Plots **True Positive Rate** (y-axis) against **False Positive Rate** (x-axis) as you vary the classification threshold.

What is AUC?

Area Under the Curve – a single number summarizing overall model performance. Ranges from 0 to 1.

AUC Interpretation:



Key insight: The closer the curve hugs the top-left corner, the better the model. A curve on the diagonal is no better than random guessing.

Running Your Code

Choose your environment: local development or cloud-based notebooks



Local IDE

VS Code, PyCharm, Jupyter

- Full control over environment
- Works offline
- Better for large projects
- Requires local Python setup

```
pip install pandas scikit-learn
```



Cloud Notebooks

Google Colab, Kaggle Notebooks

- No setup required – browser-based
- Free GPU/TPU access
- Easy sharing and collaboration
- Great for learning and prototyping

```
colab.research.google.com
```



TODAY'S WORKSHOP

We'll use Google Colab – no installation needed! Just open the link and start coding.